

```
/**
 * server/lib/production-guard.ts
 *
 * Centralized production automation guard.
 * Import and call isProductionAutomationAllowed() in every background
 * service before any automation action (AI, YouTube, vault, growth, etc.).
 *
 * Replaces per-service phantom user sets in stuck-scheduler-recovery.ts,
 * dead-letter-drain.ts, and all future background workers.
 */
```

```
// — Blocked user IDs —————
```

```
const BLOCKED_USER_IDS = new Set<string>([
  'google_api_demo_reviewer',
  'dev_bypass_user',
  'phase1-done-criteria-user',
  'phase1_test_action',
  'tiktok_-000hfxLzkfKJGE24wvR-qZP9Pw6iwxLWyeM',
  '54374239',
]);
```

```
// — Blocked YouTube channel IDs —————
```

```
const BLOCKED_CHANNEL_IDS = new Set<string>([
  'UCdemo_ETGaming247',
  'UC_test123',
]);
```

```
// — Blocked platforms —————
```

```
const BLOCKED_PLATFORMS = ['tiktok', 'rumble', 'twitch', 'kick'] as const;
```

```
// — Name-pattern checks —————
```

```
const BLOCKED_NAME_PATTERNS = ['demo', 'test', 'reviewer', 'seed', 'placeholder'];
```

```
function matchesBlockedPattern(value: string): boolean {
  const lower = value.toLowerCase();
  return BLOCKED_NAME_PATTERNS.some(p => lower.includes(p));
}
```

```
// — Public API —————
```

```
export interface ProductionGuardResult {
  allowed: boolean;
  reason: string;
}
```

```

/**
 * Call before ANY production automation for a user/channel.
 * Returns { allowed: false } for demo, test, reviewer, seed, phantom,
 * placeholder, non-YouTube platform, or invalid channel IDs.
 *
 * Usage:
 *   const guard = isProductionAutomationAllowed(userId, channelId, platform);
 *   if (!guard.allowed) {
 *     log.debug(`[ServiceName] Skipping: ${guard.reason}`);
 *     continue;
 *   }
 */
export function isProductionAutomationAllowed(
  userId: string,
  channelId?: string | null,
  platform?: string | null,
): ProductionGuardResult {
  // Blocked exact user IDs
  if (BLOCKED_USER_IDS.has(userId)) {
    return { allowed: false, reason: `Blocked user ID: ${userId}` };
  }

  // Name-pattern match on user ID
  if (matchesBlockedPattern(userId)) {
    return { allowed: false, reason: `User ID matches blocked pattern: ${userId}` };
  }

  // Blocked YouTube channel IDs
  if (channelId) {
    if (BLOCKED_CHANNEL_IDS.has(channelId)) {
      return { allowed: false, reason: `Blocked channel ID: ${channelId}` };
    }
    if (channelId.startsWith('UCdemo') || matchesBlockedPattern(channelId)) {
      return { allowed: false, reason: `Placeholder/demo channel ID: ${channelId}` };
    }
  }

  // Blocked non-YouTube platforms
  if (platform) {
    const platformLower = platform.toLowerCase();
    if (BLOCKED_PLATFORMS.some(p => platformLower.startsWith(p))) {
      return { allowed: false, reason: `Non-YouTube platform: ${platform}` };
    }
  }

  return { allowed: true, reason: 'ok' };
}

```